RESEARCH / SYSTEMS / AGENTS

大模型系统与智能体工程

专家均衡、元数据治理、请求调度与可验证执行

陶壮 Zhuang Tao

2026 年 11 月

五项工作分别回答资源分配、状态治理与执行验证问题

方向	核心问题	本次介绍的工作
MoE 专家均衡	有限副本怎样缓解设备热点	静态排布、动态调整与运行时集成
元数据平台	怎样让资产可发现、可授权、可调用	注册与治理扩展、元数据 Agent
推理请求调度	缓存复用与排队等待如何权衡	会话路由与状态刷新
服务仿真器	如何分开验证路由逻辑与时延模型	策略复用、事件模拟与条件回放
Agent 工程	多 worker 怎样协作、怎样改进执行框架	DELM 适配与 Harness2 搜索评测

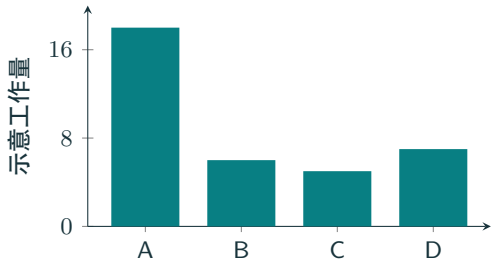
这些工作属于不同项目；下文按问题分别介绍，不预设它们已集成为一套部署系统。

MoE 的稀疏激活把单轮时延推向最忙的设备

专家数量相同，工作量仍可能悬殊

- 路由到逻辑专家的频率随输入变化。
- 热门专家副本过少，会集中计算压力。
- 复制权重消耗容量；布局还受通信与唯一性约束。

$$T_{\text{layer}} \approx \max_{d} T_d + T_{\text{comm}}$$



图中数据仅解释拖尾机制，不是硬件测试。OmniPlacement 是 OmniInfer 团队系统中的专家排布与均衡能力。

副本分配先降低单副本压力，再处理设备组合

固定预算的整数资源分配

$$\min_{r} \max_{e} \frac{a_e}{r_e}$$

$$1 \le r_e \le u_e, \quad r_e \in \mathbb{Z}, \quad \sum_e r_e \le E + B$$

- a_e：固定窗口内专家工作量。
- r_e：副本数；B：额外副本预算。
- 每次给当前 a_e/r_e 最大的专家增加副本。

专家	a_e	初始 r_e	分配后 r_e
1	12	1	2
2	8	1	2
3	4	1	1
4	4	1	1

最大单副本压力：$12 \to 6$。

这一步的最优性不等于设备布局全局最优。

条件：固定非负工作量、等成本副本、均匀分担与整数上限。堆分配时间 $O(E + B \log E)$。例子为流程示意。

设备放置同时满足容量、同卡唯一性与负载目标

$$x_{de} \in \{0,1\}, \quad \sum_{d} x_{de} = r_e$$

$$\sum_{e} x_{de} \le C_d, \qquad L_d = \sum_{e} x_{de} \frac{a_e}{r_e}$$

$$\min_{x} \max_{d} L_d$$

副本按压力降序放置，在可行设备中选择较低负载目标。

拓扑重排进一步分散主机间热点，采用具体分组规则。

设备 A：$e_1(6) + e_2(4) + e_3(4)$

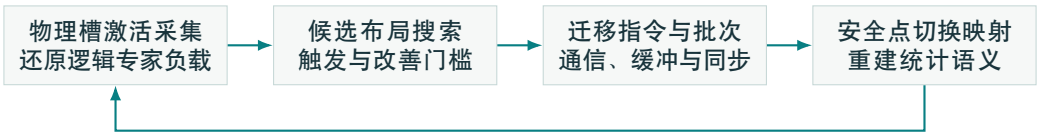
设备 B：$e_1(6) + e_2(4) + e_4(4)$

每设备 3 个槽位；同一专家不重复落在同一设备。 $L_A = L_B = 14$

贪心可能因顺序而构造失败；可行性和负载质量分别检查。

示例沿用上一页工作量；原静态函数不是任意拓扑的最优求解器。公开复现中的最大流回退仅保证合法布局。

动态均衡把采集、候选决策与权重更新连成闭环



- 统计必须对应当前布局，多个副本要汇总到同一逻辑专家。
- 当前不均衡达到触发阈值、候选改善超过裕度后才更新。
- 运行时采用布局代理收益；热点持续期和复制时间的成本判据在模拟中另行建模。

运行时链条按项目能力介绍；本人重点贡献包括优化器、工具链、批量传输、测试和多模型适配。

迁移按冲突拆批，并在权重就绪后更新映射

控制面产生差异指令

1. 比较当前布局与目标布局。
2. 将新增、移除与传输指令分组。
3. 拆分存在 rank 冲突的批次。

数据面控制可见状态

1. 后台通信写入缓冲并同步完成状态。
2. 主线程在安全点把就绪数据复制到设备。
3. 更新变更层的 selector，清理标志并切换统计口径。



OmniPlacement / Omni-EPLB 运行时设计摘要。未把整套框架、所有通信机制或系统性能归为个人独立成果。

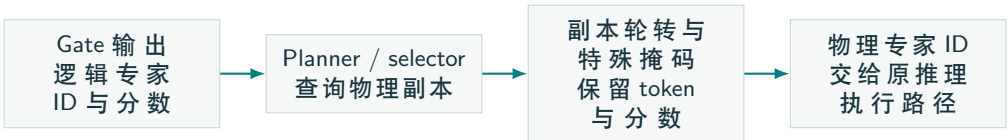
Pattern 工具链把激活记录转换为可检查的部署文件



工程环节	具体实现与检查对象
输入契约	区分 prefill / decode、统计步区间、层数与专家数，核对采集位置与专家编号的关系
输出产物	rank × layer × expert 的部署文件，以及基线/候选负载 CSV 和可视化
接入检查	工具检查专家覆盖与各 rank 数量一致性；Planner 启动时检查并行规模匹配

本人工作包括 Pattern 流水线、CLI/YAML、适配和测试。覆盖检查不等于所有约束都已独立验证；完整交付宜同时保存采集窗口与配置。

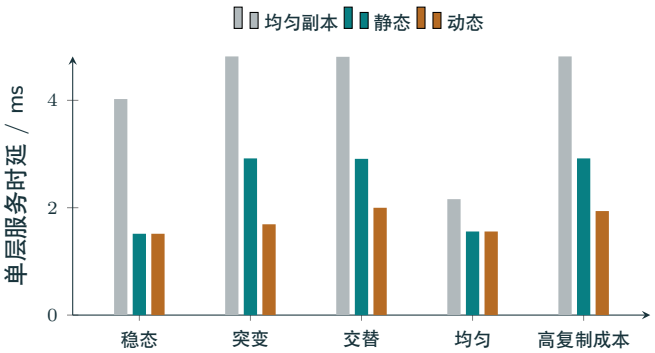
框架适配保留逻辑专家选择，并转换物理执行位置



接口或适配点	实现职责
`plan()`	使用传入的当前层映射，将逻辑专家转换为物理位置
`record_activation()`	采集物理位置上的激活，供统计与候选决策使用
`place_experts()`	在主路径安全点应用已就绪权重与映射更新
模型差异	层索引、稠密前缀、共享 / zero expert 和 Pattern 维度分别适配

本人参与接口、配置、多模型与特殊专家适配。这里不改变 Gate 的逻辑选择，不推定所有框架和模型均已完成同等实测。

合成模拟中，热点变化决定动态更新的价值



动态相对静态

稳态偏斜：0.0%
热点突变：-42.1%
高复制成本：-33.6%

过去窗口预测；更新代价计入同步复制。

不是 GPU/NPU 或完整模型测速。

来源：results/reference/eplb_aggregate.csv；3 个固定种子均值，360 步，逻辑激活与显式时间模型。

元数据平台把资产注册、授权和消费连接起来



- **注册约束**：统一数据集标识、责任主体与不可变字段。
- **授权语义**：身份、全局/领域角色与运行时策略保持一致。
- **平台适配**：扩展 API、GMS、登录链路和前端操作共同演进。

本人负责相关企业扩展与前后端适配；DataHub 是上游平台。此图为通用功能示意，不披露内部拓扑或部署信息。

注册 SDK 用分层接口连接资产建模、读写和验证



工程决策	作用与边界
计算与 I/O 分离	URN / Aspect 可独立校验；搜索走 ES，存在性和精确读取走 GraphQL
注册约束集中	服务端注入主体、检查域与依赖；同 URN 的注册用进程内锁协调
发布与查询分开	REST / Kafka 按路径选择；发送后轮询确认可查询，超时显式返回

本人负责扩展重构、API/SDK 与注册约束。发布成功不等于索引已可查询；进程内锁和轮询确认不构成跨副本事务。

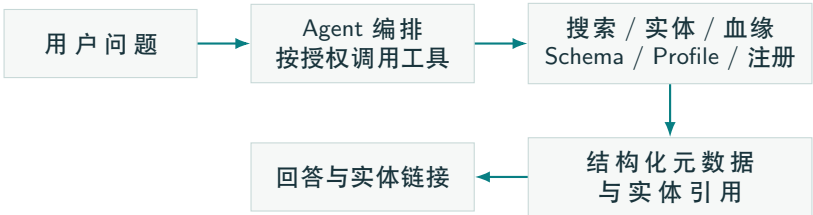
身份与角色规则必须贯穿写入和读取链路

检查层次	工程约束	解决的问题
身份与准入	登录后建立可追踪的访问主体	前端登录与后端鉴权一致
领域与角色	按领域/全局范围解析角色	资产边界明确、策略可解释
操作权限	注册、管理、发布分别受控	管理权限不会自动泛化为所有写权限
凭证衔接	API 与界面采用一致的授权语义	避免多入口规则分叉

个人实现范围：注册 SDK、扩展服务重构、GMS 认证策略、前端与测试。

此处保留通用治理机制；省略内部身份系统、人员信息、域名、业务规模与凭证配置。

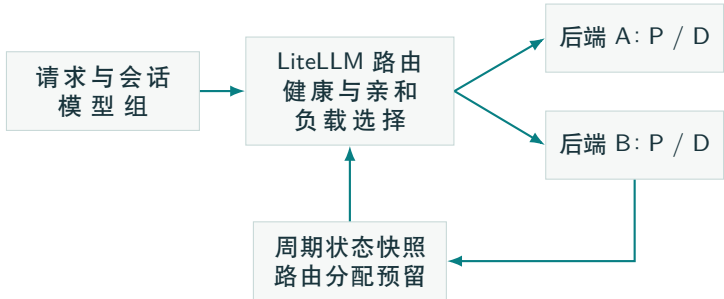
元数据 Agent 用工具结果建立可追溯的回答



- 工具访问已有元数据和权限服务，语言模型负责交互与编排。
- 返回实体链接，用户可以检查来源并继续浏览资产。
- 本人实现工具接入、结果实体链接、配置和回归支持。

这是元数据交互能力；不将其等同于开题中的模型语义解释，也不宣称自动完成数据质量评估。

请求路由需要同时考虑等待、缓存和状态新鲜度



- 保持会话有利于前缀复用，但拥堵后端可能拉长等待。
- 快照具有延迟，分配后的预留计数影响后续选择。

路由算法由本人和同事共同设计；个人实现还包括独立状态刷新与指标重复计数修复。图为逻辑架构。

算法一优先保持会话，并为新会话选择低负载候选

下表为合成复现中的统一策略口径。

请求分支	路由动作
已有健康映射	保持原后端，延续缓存复用机会
新建亲和会话	从最低负载的 $k = \max(1, \lceil 0.1N \rceil)$ 个候选中选择
无亲和要求	按请求形态进入最小负载或稳定散列路径

优点

机制直接，兼顾会话稳定性与新流量分散。

边界

已有会话遇到拥堵时，保持映射可能继续累积等待。

0.1 是该算法的候选比例，不是理论最优常数。合成模拟的统一 P/D 计数协议见 docs/MODEL.md。

算法二用等待收益与缓存损失比较决定是否切换

已有会话从 A 切换到 B 的简化判据

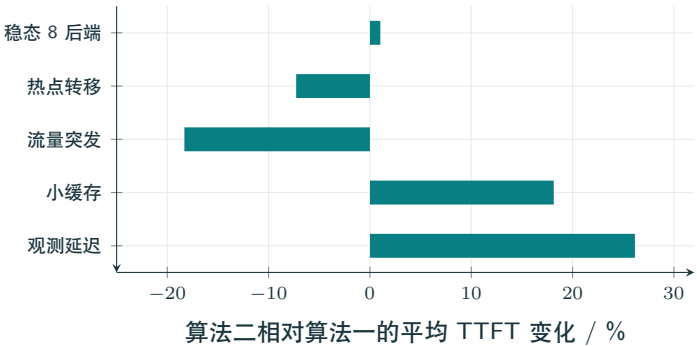
$$\underbrace{\widehat{W}_A - \widehat{W}_B}_{\text{预计减少的等待}} > \underbrace{1.2\,\widehat{\theta}_A \bar{n}/v}_{\text{原端缓存收益的时间估计}}$$

- $\widehat{W}_i$：可观测状态下的预计等待。
- $\widehat{\theta}_A$：原端历史 token 复用比例。
- $\bar{n}/v$：平均输入的预填充时间尺度。

- 两侧单位均为时间。
- 统计缺失、相等时保持会话。
- 决策前切换；不迁移在执行的请求或缓存。

这是理论分析与模拟策略。模拟不读取 B 的真实缓存，执行时仍计算实际命中；1.2 的裕度不构成预测误差保证。

缓存容量与状态延迟会改变切换策略的收益方向



同一策略存在收益与退化

突发：-18.31%
小缓存：$+18.14\%$
观测延迟：$+26.15\%$

负值为变快，正值为变慢。目标端缓存与预测误差不能忽略。

来源：results/reference/routing_aggregate.csv；3 固定种子、合成请求、显式 P/KV/D 时间模型；非真实服务性能。

状态刷新与计量准确性是路由可用性的基础

独立刷新进程

- 周期采集服务状态并更新共享存储。
- 使路由读取与状态刷新职责清晰。
- 部署和异常处理围绕可观测状态展开。

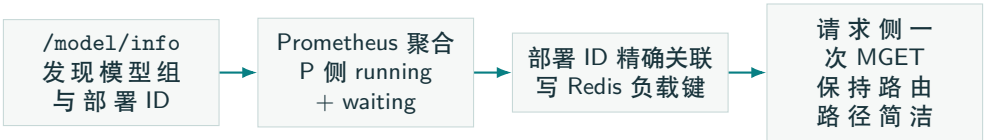
Prometheus 计量修复

- 检查 pipeline key 请求的计数链路。
- 修复重复计数，恢复指标解释的一致性。
- 将状态语义和路由行为一起核对。

本人负责上述实现；路由方案按共同设计成果介绍。

外部交流版省略内部域名、原始配置、业务流量与未公开验收数据。

部署发现与稳定标识连接减少负载状态的静默漏刷



异常或边界	具体处理
running / waiting 单边存在	按实例聚合后外连接，保留单边记录，再按集群求和
部署未匹配监控标签	显式列出漏刷项，区分缺部署与疑似命名脱节
配置发现接口不可用	可显式采用配置降级，并在日志注明实际来源

本人直接实现独立刷新链路。P 侧请求数是代理负载；刷新时间与状态年龄判断仍需补强，不把它等同剩余 token 工作量。

服务仿真器将路由决策与后端执行成本分离



- 复用策略代码，降低仿真与服务逻辑分叉的风险。
- 用可替换服务模型研究请求在途、亲和与耗时假设的影响。
- 分别检查决策等价、计时合理性和外推有效性。

详见 docs/simulator-technical-report.md。此服务仿真器与本仓库 llm_repro 的数学复现模型分别维护。

统一数据契约使采集、插件与服务后端可以分别替换

接口边界	输入与输出	工程检查点
TraceRecord	请求 ID、到达、token、E2E、模型组；落点 / TTFT 可选	采集时校验字段，保留路由键的完整语义
路由插件	请求、健康候选与状态适配；返回目标部署	直接加载原模块，用不对称探针检查选点分支
服务 Backend	请求规模、当前在途与命中近似；返回 E2E / TTFT	参数化与经验后端共享调用接口，耗时单位明确
运行归档	运行元数据、指标与种子结果	记录插件指纹；完整拟合参数与输入哈希仍需补齐

插件导入、路由真正生效、服务模型有效，是三个可以分别诊断的问题。

本人实现模块拆分、Backend 解耦、采集与插件调用修正。契约兼容不意味着任意插件的外部依赖已经被完整仿真。

批量读写、变化集同步与内存适配降低状态访问开销

保留相同状态口径，减少重复工作

1. 将多键状态读写合并为批量操作。
2. 在完成与派发后跟踪变化集群，减少全量重复同步。
3. 在适用场景用进程内存储承接高频状态调用。

检查优化后的接口行为

- 比对同步后的在途计数和插件落点。
- 分开记录模拟时间与真实执行墙钟。
- 单列真实 Redis 与内存模式的语义覆盖。

模式	当前边界
真实 Redis	键过期受墙钟影响，需要核对快速模拟时钟与 TTL 的关系
内存适配	未完整实现 TTL / Lua / 原子语义，适合匹配接口的受控扫描

这些是已实现的开销优化路径；本次未新增性能测速，不给出未经测量的加速倍数或生产收益。

开环回放与闭环容量回答不同的实验问题



模式	到达方式	适用问题
开环回放	保留输入到达过程	给定流量下的状态和策略行为
闭环容量	固定并发水位，完成后补位	指定水位与模型下的完成率

服务后端返回请求级耗时，未逐 token 模拟 P/D、KV 链路或连续批处理；这些细化机制属于 llm_repro 参考模型。

条件回放与反事实预测需要不同的验证证据

模式	已知信息与用途	能够支持的结论
条件回放	注入真实落点及观测时序；检查原路径	给定条件下的决策与记录一致性
经验后端	用观测样本或拟合关系描述执行成本	已覆盖负载范围内的条件估计
反事实策略对照	重算落点、在途与会话命中近似	需要独立验证新分布下的计时与排序

将真实时延带回回放，可以核对链路；它不能证明新策略的时延预测准确。

共享同一个后端模型也不自动保证策略排序正确：策略改变会反过来改变会话命中、在途和并发分布。

验证矩阵分别约束策略逻辑、计时模型与校准方法

验证层	检查对象	当前可报告的边界
策略接入	相同输入、状态和随机性下的分支	状态协议、计数更新和落点分别比对
事件执行	请求派发、完成顺序与在途计数	事件与计数性质优先于均值拟合
校准自检	有信号 / 无信号 / 反向 / 退化输入	合成自检 4/4 通过
外推评估	未参与校准的负载与策略	留出工作负载，检查误差和排序稳定性

内存存储替身并未完整实现 Redis 的 TTL、Lua 和原子语义。

本轮实际运行 calibrate-selftest：4 类合成测试全部通过。这是方法与退化输入检查，不是线上收益或预测精度。

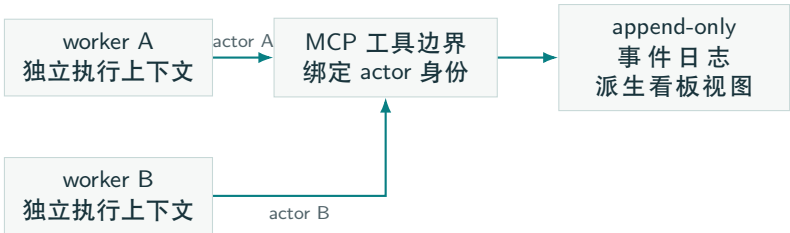
两类模拟分别服务机制解释与服务代码验证

	llm_repro 参考模型	服务 simulator
主要目的	固定条件下解释算法取舍	复用策略链路并检验服务行为
输入	可公开的合成轨迹与固定种子	合成输入或授权的脱敏轨迹
执行成本	显式 P/KV/D 阶段、有限前缀缓存	请求级耗时模型、会话命中近似
对外证据	原始记录、参数和退化结果	接入机制、验证方法和合成自检

后续验证重点：固定轨迹做策略成对比较，独立留出校准数据，检查缓存与排队分布变化。

复现模型没有代替真实设备测量；服务仿真器也不能凭回放一致率替代预测验证。

DELM 适配把双 worker 协作组织成可追溯的看板状态



- 每个 worker 同时最多持有一个 claim，显式记录任务状态。
- 进程内事件唤醒等待者，减少对看板的盲目轮询。
- 本人完成相关 JavaScript 运行时适配、工具桥接与会话组织。

DELM 沿用上游框架称谓，不扩展缩写；看板协议来源与本地适配分别归因，不宣称个人原创整个协议。

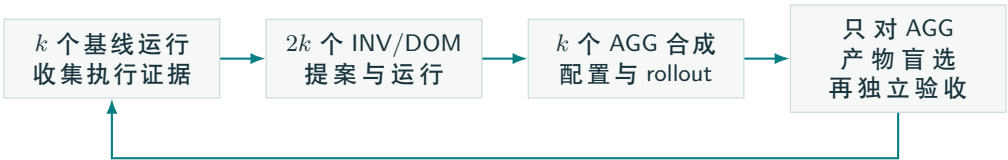
日志、claim 与文件哈希共同约束协作一致性

机制	约束	工程意义
事件追加	状态由日志归约为视图	可以追踪任务认领和状态变化
单 worker 单 claim	认领与完成显式执行	避免一个 worker 同时占据多项工作
actor 绑定	工具调用的主体由会话绑定	不依赖模型自行填对身份
文件 apply 校验	用 SHA-256 检查预期基础内容	发现已变化的文件，拒绝冲突写入

进程内事件与文件冲突检查不等于跨进程事务或任意并发条件下的形式化保证。

哈希保护仅覆盖 board.apply；worker 直接编辑共享文件仍可能冲突。公开报告不附源项目代码与配置。

Harness2 将执行框架变体纳入真实 rollout 的搜索循环



- 被优化的是执行系统的可变部分，而不只是一段静态提示词。
- 并行模式最终选择池不包含基线或中间变体，没有内建弃选。
- 本人负责 Harness2 运行时适配、可靠执行和 Agent 接线；搜索方法源于上游。

图示为上游并行搜索流程；顺序模式另有候选链。选中候选不证明优于基线，需要独立任务测试。

执行轨迹把候选配置、实际运行与候选选择连接起来



- 候选描述和实际应用配置应一致，避免只改变标签而没有改变执行。
- rollout_ok 表示运行可分析；进入候选选择还要有可读产物。
- 结果盲评用于减少候选身份的影响；仍需留出任务与多次运行控制评测偏差。

DELM 适配提供执行机制，Harness2 提供搜索与评测组织；不将二者合称为一个未经验证的全新协议。

版本化 runner 将候选配置落到真实执行和可读结果

工程接口	已实现的接线与产物
调用输入	任务目录、harness 目录、run ID、模式和超时；运行标识进入路径前校验
配置物化	合并系统提示与规则，映射技能/工具指南/子代理，注入两 worker 启动提示和看板指导
进程协议	stdout 只返回单行 JSON，stderr 承载进度；原始事件转换为统一轨迹文本
结果目录	`trajectory.txt`、`events*.jsonl`（双 worker 分别记录）、`workspace/`、`meta.json` 与 runner 报告

执行状态、工具活动、候选产物分别保留；无法获得的成本写为空值，不用零值代替。

本人负责执行桥接、配置接线和轨迹转换；上游提供搜索组织方法。runner 契约通过不等于模型任务已通过。

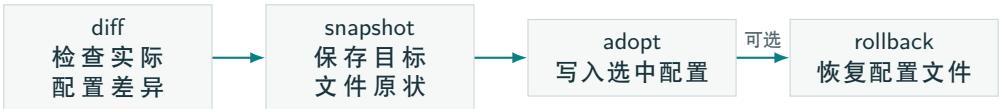
可靠交付需要分开判断运行、产物与任务正确性

机制	本人实现或修正	避免的问题
输入 staging	任务文本哈希、输入额度与存储根配置	不同任务错误复用输入或遗漏必要文件
运行作用域	用户/会话命名空间、重跑工作区重建	并发覆盖和上次产物残留
产物识别	比较 seed 内容，保留新增或已改文件	修改原文件却被误判为无交付
停滞处理	逐 worker 观察、提醒与失败窗口	活跃 worker 掩盖静默同伴

正常退出、存在工具活动、有候选产物、通过任务验收，是四个不同判断。

输入哈希并非完整项目内容寻址；运行目录存在也不证明 Agent 已执行。采纳配置与回收工作产物分别处理。

配置采纳与任务产物回收分开，支持审阅和文件级恢复



配置操作

- 采纳前比较物化后的实际映射。
- 恢复旧内容，删除本次新增的配置文件。

独立回收 workspace

- 提取新增或内容改变的任务文件。
- 排除配置和记账文件；过滤项列出清单。

这是本人承担的结果管理与可靠交付工程。rollback 仅恢复采纳涉及的配置文件，不回滚 Agent 的全部项目修改，也不提供整项目事务。

离线回归已覆盖核心机制，完整集成仍有明确缺口

验证对象	本轮实际结果	可以支持的结论
Board 单元测试	13/13 通过	看板关键状态与操作符合用例
Harness2 JavaScript 测试	三套合计 58/58 通过	配置物化、runner dry-run 契约与轨迹渲染
Python 全量发现	84 项：58 通过、9 失败、12 错误、5 跳过	旧任务接口和 seed fixture 与当前实现存在差异
会话编排单元测试	所需 Agent SDK 缺失，加载终止	此项未执行；真实模型端到端实验另需开展

通过单元测试不等于多任务成功率提升；不宣称完整测试全绿。

基于已提交版本的独立快照执行；未将源工作区未提交变更作为结论依据。详见 docs/delm-harness2-technical-report.md。

个人贡献以具体模块与可核验证据界定

方向	本人工作	团队、上游或验证边界
OmniPlacement	算法实现、运行时优化、工具与测试	OmniInfer 系统与论文按团队成果归因
元数据平台	SDK、扩展 API、GMS/前端适配、Agent	DataHub 提供上游平台能力
请求路由	共同设计路由；独立刷新与计量修复	不将整体网关收益拆成个人单机制收益
Simulator	策略接入、执行模拟与验证工具链	回放一致性与反事实精度分别验证
DELM / Harness2	运行时适配、协作工具、搜索评测	DELM 上游来源保留；端到端收益待验证

Simulator、DELM 与 Harness2 结合个人职责、代码历史与本轮验证整理；上游协议与搜索方法保留来源归属。

原研究目标已形成工程产出，探索项保持明确边界

原目标	实际完成	范围变化与边界
专家排布与通信亲和	Pattern、受约束放置、主机间重排	实作扩展到计算热点、容量和拓扑约束
结合专家均衡改进算法	静态副本、动态采集与更新运行时	将算法与统计、迁移、安全切换共同实现
形成可用算法与加速能力	独立 SDK、多模型适配、可复现实验	将工程能力与团队系统效果分别验证
专家预读取与语义刻画	未形成独立可核验成果	原探索项保留；不以元数据 Agent 替代

Simulator 与 Agent 工程拓展了验证和执行工具；它们不倒写为原开题的既有成果。

原始目标对应开题中的专家亲和、部署代价、专家均衡、预读取与语义研究；不推定所有探索均已完成。

后续研究聚焦观测误差、仿真外推与 Agent 评测稳定性

可讨论的问题	下一步可检验的方法
何时值得改变已有资源或会话映射	将迁移、缓存损失和观测误差统一为时间成本
如何判断模拟给出的策略排序可靠	留出轨迹、跨负载校准、成对对照与排序反转分析
如何比较不同 Agent harness	固定任务集、结果盲评、执行预算和多种子重复

可交流方向：推理系统、可验证仿真、多 Agent 执行与评测

复现材料：https://github.com/taostronger/llm-mock-1。下附技术细节与参考材料。

附录：单副本子问题的性质不能外推到设备最优

- 给定阈值 $t > 0$，专家 e 至少需要 $\lceil a_e/t \rceil$ 个副本，并满足 $1 \le r_e \le u_e$。
- 因此阈值可行性由各专家最低副本数、上限和总预算共同决定。
- 给当前最大 a_e/r_e 增加副本，求解固定等成本模型下的单副本瓶颈。
- 设备目标 $\max_d \sum_e x_{de} a_e/r_e$ 还包含组合与容量限制；需单独求解。

阶段	时间复杂度
副本分配	$O(E + B \log E)$，heapify 加 B 次堆更新
贪心放置	$O(M \log M + MN + NE)$，$M = \sum_e r_e$

均匀分流、固定工作量、等成本是性质前提。放置空间含 $N \times E$ 映射矩阵，不是仅 $O(E)$。

附录：请求模型明确区分统计估计与执行时缓存命中

路由决策读取历史状态

- 0.25 s 状态采样。
- 最近 120 个 P 完成请求的 token 复用比。
- 缺统计时保持原会话。
- 分配预留与已发布计数共同参与选择。

实际执行重新检查缓存

- 独立 P 缓存，默认 8192 个 32-token 块。
- 最长连续前缀；首次 miss 后停止复用。
- P 完成后写入有限 LRU。
- 会话保持不直接等于缓存命中。

1800 个开环请求；前 20% 预热；种子 31/47/89。P95 汇总为各次 P95 的均值。

完整参数与 TTFT 首个 D token 边界见 docs/MODEL.md。速率未由真实设备标定，有限轨迹不自动表示稳态。

附录：排队分析用于解释等待，不替代 P/D 事件执行

每个后端的 M/G/1 参考近似

$$\rho_i = \lambda_i \mathbb{E}[S_i] < 1, \qquad \mathbb{E}[W_i] = \frac{\lambda_i \mathbb{E}[S_i^2]}{2(1-\rho_i)}$$

- 平均服务时间相同，服务时间方差不同，等待仍可能不同。
- 利用率接近 1 时，预测误差对等待的影响显著放大。
- 缓存改变服务分布；会话路由也会改变到达分流。
- 泊松到达、独立服务、FCFS 等条件限制此解释；连续批处理不严格满足该模型。

本项目分别使用排队理论解释机制、离散事件模型研究具体执行过程；不把多个后端当成一个服务台。

附录：资料与复现入口

1. OmniInfer 团队论文：*System-Wide Acceleration Techniques for Optimizing LLM Serving Throughput and Latency*.
   `https://arxiv.org/abs/2511.22481`
2. Omni-EPLB 公开项目与原静态函数来源：
   `https://gitee.com/omniai/omni-eplb`
3. 合成实验、固定参数与逐请求记录：
   `https://github.com/taostronger/llm-mock-1`
4. 本仓库文档：`docs/MODEL.md`、`docs/ALGORITHMS_SCENARIOS_REPORT.md`。
5. 新增专题：`docs/simulator-technical-report.md`、`docs/delm-harness2-technical-report.md`。
6. Agent 方法来源：上游 Delm 协作语义、上游 Harness^2 搜索框架（Apache-2.0）；本文介绍运行时适配，不分发其代码。

本演示稿只包含原创概述、通用机制和公开合成结果；未分发 Agent 源项目代码、内部轨迹或部署配置。